แผนแม่บทสถาปัตยกรรม Hybrid Multi-Source Model Hub

Architecture Plan: Existing Central AI Gateway (LiteLLM) + Direct External Providers Integration

วันที่จัดทำ: 3 พฤศจิกายน 2026
กลยุทธ์หลัก: Central Gateway + Direct Providers (No Local Weights)
ความปลอดภัย: AES-256 + ZERO-KEY LEAKAGE

บทสรุปเชิงกลยุทธ์ (Strategic Summary): เพื่อตอบโจทย์ความยืดหยุ่นสูงสุดโดยไม่สร้างภาระด้าน Infrastructure ระบบ AI Builder จะใช้สถาปัตยกรรม **Hybrid Multi-Source Model Hub** โดยดึงความสามารถของ **Central AI Gateway (LiteLLM)** ที่มีอยู่แล้วในองค์กรมาเป็นแกนหลัก (รองรับ Dynamic Model Discovery, Auto-Failover, และ Token Quota Tracking) พร้อมเปิดช่องทางให้สามารถเชื่อมต่อ **Direct Model จาก Provider ภายนอกอื่นๆ** (เช่น Google Gemini Direct, OpenAI, Anthropic, Azure, Groq, หรือ Custom HTTP) ได้อย่างอิสระ โดย **ไม่มีการติดตั้ง LiteLLM Server ซ้ำซ้อน และไม่มีการจัดเก็บไฟล์ Local Model Weights** ในระบบ

แหล่งโมเดลหลัก (Primary)	แหล่งโมเดลเสริม (Direct)	ภาระเซิร์ฟเวอร์	ระบบสิทธิ์ & คีย์ลับ
AI Gateway	**Multi-Provider**	**Zero Weight**	**AES-256**
LiteLLM Proxy กลางขององค์กร	Google / OpenAI / Anthropic / Groq	ไม่มีการโหลดไฟล์โมเดลลงดิสก์	RBAC + Masked Key 100%

1. สถาปัตยกรรมการเชื่อมต่อแบบไฮบริด (Hybrid Architecture Overview)

ระบบ AI Builder จะทำหน้าที่เป็น **Unified Model Orchestrator** ที่จัดเก็บการตั้งค่าโมเดลไว้ใน PostgreSQL และส่งต่องานไปยังเป้าหมายอย่างแม่นยำ:

แหล่งกำเนิดโมเดล (Source)	การกำหนดค่าและการยืนยันตัวตน	ฟีเจอร์เด่นที่ดึงมาใช้งาน	บทบาทใน AI Builder Platform
1. Central AI Gateway (Existing LiteLLM)	• กำหนด `GATEWAY_URL` ในระบบ • ใช้ Service Virtual Key ตัวเดียว • Zero-Key Leakage ต่อค่ายภายนอก	• **1-Click Dynamic Sync:** ดึงโมเดลใหม่ผ่าน `/v1/models` • **Smart Failover:** สลับค่ายอัตโนมัติเมื่อ Rate Limit • **Spend Analytics:** บันทึกสถิติต้นทุนแยกรายคน/แผนก	**เสาหลักสำหรับงานองค์กร:** ทุกคนในทีมสามารถเลือกโมเดลมาตรฐาน (Gemini, GPT-4o, Claude) ได้ทันทีโดยไม่ต้องมี API Key ส่วนตัว
2. Direct External Providers (Google / OpenAI / อื่นๆ)	• กรอก API Key ของ Provider ตรง • เข้ารหัส AES-256 ลง PostgreSQL • ระบุ Base URL (กรณี Custom/Azure)	• **ความอิสระสูงสุด:** ต่อโมเดลรุ่นใหม่ล่าสุดได้ทันที • **BYOK Support:** ผู้ใช้นำคีย์ส่วนตัวมาใช้งานได้ • **Custom HTTP:** ต่อระบบ AI ภายในเฉพาะทีมได้	**ความยืดหยุ่นเฉพาะกิจ:** สำหรับผู้ที่ต้องการใช้โมเดลรุ่นทดลอง (Preview/Experimental) หรือโมเดลพิเศษที่ยังไม่ได้ใส่ไว้ใน Gateway กลาง

2. การออกแบบโครงสร้างฐานข้อมูลรวมศูนย์ (PostgreSQL Schema)

จัดเก็บโมเดลทุกแหล่งไว้ในตารางเดียว `ai_models` โดยใช้ฟิลด์ `source_type` เป็นตัวจำแนกการทำงาน:

```
class AIModel(Base): __tablename__ = "ai_models" id = Column(String(36), primary_key=True, default=lambda: str(uuid.uuid4())) source_type = Column(String(20), nullable=False) # 'gateway' หรือ 'direct_provider' provider = Column(String(50), nullable=False) # 'gateway', 'google', 'openai', 'anthropic', 'groq', 'custom_http' model_identifier = Column(String(100), nullable=False) # e.g. 'gemini-2.0-flash', 'gpt-4o', 'claude-3-5-sonnet' display_label = Column(String(150), nullable=False) # ชื่อที่แสดงบน Canvas endpoint_url = Column(String(255), nullable=True) # Base URL เฉพาะกรณี Custom HTTP หรือ Azure encrypted_api_key = Column(Text, nullable=True) # คีย์ลับเข้ารหัส AES-256 (ว่างได้หากใช้ Gateway Key กลาง) scope = Column(String(20), default="system") # 'system' (ทุกคนใช้ได้) หรือ 'personal' (เฉพาะเจ้าของ) owner_id = Column(String(36), ForeignKey("users.id"), nullable=True) context_window = Column(Integer, default=128000) supports_vision = Column(Boolean, default=False) is_enabled = Column(Boolean, default=True) # เปิด/ปิดการแสดงใน Canvas Dropdown is_default = Column(Boolean, default=False) # โมเดลเริ่มต้นเมื่อสร้าง Agent ใหม่
```

กลไกการทำงานของระบบและส่วนประกอบทางเทคนิค

3. กลไกการดึงข้อมูลและ Sync โมเดลจาก Central AI Gateway

One-Click Dynamic Discovery API

เมื่อผู้ใช้กดปุ่ม **Sync Models from Gateway** ในหน้าจอ:

- Backend ยิงคำขอ `GET {GATEWAY_URL}/v1/models` โดยใช้ Bearer Service Key กลาง
- นำข้อมูลโมเดล (ID, Context Window, Provider Tag) มา Upsert ลงในตาราง `ai_models` (ประเภท `gateway`)
- คงค่า Preferences เดิมของผู้ใช้ไว้ (เช่น ค่า `is_enabled` หรือการตั้งเป็น Default)

Context-Aware Metadata & Budget Tracking

ทุกคำขอที่ส่งไปยัง Gateway จะแนบ Metadata เพื่อคิดต้นทุน:

```
extra_body = { "user": current_user.username, "metadata": { "workflow_id": state["workflow_id"], "run_id": run_id, "department": current_user.department } }
```

Gateway จัดเก็บสถิติต้นทุนแยกรายคน/แผนก และคุม Budget ไม่ให้เกินโควต้า

4. กลไกการประมวลผลคำขอ (Unified Execution Dispatcher)

ใน `backend_fastapi/app/graph_engine/nodes/agent.py` ตัวแทน Agent จะตรวจสอบแหล่งกำเนิดโมเดลและเลือกส่งคำขออย่างถูกต้อง:

```
# ตรวจสอบประเภทโมเดลที่เลือกใช้งาน model_cfg = await get_ai_model_by_id(model_id) if model_cfg.source_type == "gateway": # ส่งเข้า Central AI Gateway (LiteLLM) พร้อม Metadata สำหรับคิด Token client = AsyncOpenAI(base_url=GATEWAY_URL, api_key=GATEWAY_KEY) response = await client.chat.completions.create(model=model_cfg.model_identifier, ... ) elif model_cfg.source_type == "direct_provider": # ถอดรหัสลับคีย์ AES-256 ประจำโมเดลนั้น decrypted_key = decrypt_secret(model_cfg.encrypted_api_key) if model_cfg.provider == "google": # เรียกผ่าน Gemini OpenAI-compatible หรือ Google SDK โดยตรง client = AsyncOpenAI(base_url="https://generativelanguage.googleapis.com/v1beta/openai/", api_key=decrypted_key) else: # OpenAI, Anthropic (ผ่าน Adapter), Groq, หรือ Custom HTTP URL client = AsyncOpenAI(base_url=model_cfg.endpoint_url, api_key=decrypted_key) response = await client.chat.completions.create(model=model_cfg.model_identifier, ... )
```

5. สถาปัตยกรรมความปลอดภัยและการปกป้องคีย์ลับ (Security & Encryption)

มาตรการความปลอดภัย	วิธีการปฏิบัติในระบบ	ประโยชน์ด้านความมั่นคงปลอดภัย
Encryption at Rest	ใช้ `cryptography.fernet` หรือ `AES-256-GCM` ร่วมกับ `APP_ENCRYPTION_KEY` ที่เก็บใน `.env`	หากไฟล์ฐานข้อมูลถูกขโมย ผู้โจมตีจะไม่สามารถอ่านค่า API Key ของผู้ใช้ได้
Zero-Key Frontend Leakage	Endpoint `GET /api/models` จะ Mask คีย์เป็น `••••••••••••` เสมอ ไม่ส่ง Plaintext ออกไป	ป้องกันการดักจับคีย์ผ่าน Browser DevTools หรือ Network Inspection
RBAC Scope Isolation	โมเดลประเภท `personal` จะถูกมองเห็นและใช้งานได้เฉพาะผู้ที่เป็น `owner_id` เท่านั้น	ผู้ใช้คนอื่นไม่สามารถนำคีย์ส่วนบุคคลของผู้อื่นไปใช้รันงานได้

หน้าจอการใช้งาน แผนการดำเนินงาน และตัวชีวัด

UI Design, Phased Implementation Milestones, and Success KPIs

6. การออกแบบหน้าจอผู้ใช้ (Frontend "Models & Endpoints Hub")

ปรับปรุงหน้า `ModelsAndEndpointsPage.tsx` ตามมาตรฐาน **Builder Theme** แบ่งการทำงานเป็น 2 แท็บชัดเจน:

แท็บ 1: Central AI Gateway (องค์กร)

- **Gateway Status Bar:** แสดงสถานะเชื่อมต่อ (เช่น `Connected (42ms)`)
- **Sync Button:** ปุ่ม `[ Sync Models from Gateway ]` ดึงโมเดลใหม่ใน 1 วินาที
- **Model Switchboard:** ตารางแสดงโมเดล พร้อม Toggle สลับเปิด/ปิดการแสดงผลใน Canvas
- **Spend Card:** แสดงโควต้าและยอดใช้งาน Token เดือนปัจจุบัน

แท็บ 2: Direct External Providers (คีย์ตรง)

- **Add Model Modal:** ฟอร์มเพิ่มโมเดลตรง เลือกระหว่าง Google, OpenAI, Anthropic, Groq, หรือ Custom HTTP
- **Test Connection:** ปุ่มยิง Ping ทดสอบว่า API Key และ Endpoint ใช้ได้จริงก่อนกดบันทึก
- **Scope Selector:** เลือกว่าจะแชร์เป็น `Shared (System)` หรือเก็บไว้ใช้คนเดียว `Personal (BYOK)`

7. แผนการดำเนินงานทีละขั้นตอน (Implementation Action Plan)

ระยะงาน (Phase)	งานที่ต้องดำเนินการ (Action Items)	ไฟล์ที่เกี่ยวข้อง	เกณฑ์ความสำเร็จ (Milestone)

Phase 1: Foundation	1. สร้างตาราง `ai_models` ใน PostgreSQL
	2. พัฒนายูทิลิตี้เข้ารหัส `crypto.py` (AES-256)
	3. กำหนดตัวแปร Gateway ใน `config.py`
		`app/models/ai_model.py`
		`app/utils/crypto.py`
		`app/config.py`
			สร้างตารางใน DB สำเร็จ และฟังก์ชัน Encrypt/Decrypt ผ่านการทดสอบ 100%

Phase 2: Gateway Sync	1. พัฒนา API `POST /api/models/sync-gateway` ดึง `GET /v1/models` 2. พัฒนา API `GET /api/models` ส่งรายการโมเดล	`app/routers/models.py` `app/services/gateway.py`	สามารถดึงโมเดลจาก Gateway มาแคชใน PostgreSQL ได้อย่างสมบูรณ์
Phase 3: Direct CRUD	1. พัฒนา API `POST /api/models/direct` เพิ่มโมเดลตรง 2. พัฒนา API `POST /api/models/test-connection`	`app/routers/models.py`	สามารถเพิ่มโมเดล Google Gemini / OpenAI ตรง พร้อมทดสอบการเชื่อมต่อได้
Phase 4: Engine Dispatch	ปรับปรุง Agent Node ใน LangGraph ให้ตรวจสอบ `source_type` และยิงคำขอไปยังปลายทางที่ถูกต้อง	`app/graph_engine/nodes/agent.py`	รัน Workflow สำเร็จทั้งโมเดลจาก Gateway และโมเดลจาก Direct Provider
Phase 5: Frontend UI	ปรับปรุงหน้า `ModelsAndEndpointsPage.tsx` มี 2 แท็บ และอัปเดต Dropdown ใน Canvas Inspector	`frontend/src/components/ ...` `useAiModels.ts`	UI สวยงามสไตล์ Builder Theme และใช้งานได้ครบทุกฟีเจอร์

สรุปความพร้อมในการเริ่มลงมือทำ (Execution Readiness)

สถาปัตยกรรม Hybrid Multi-Source Model Hub นี้จะมอบความยืดหยุ่นระดับองค์กรให้กับ AI Builder Platform โดยผสานจุดแข็งของ Central AI Gateway (LiteLLM) ในเรื่องการควบคุมงบประมาณและความเสถียร เข้ากับความคล่องตัวของ Direct Providers โดยไม่สร้างภาระด้านเซิร์ฟเวอร์ พร้อมเริ่มลงมือพัฒนา Phase 1 ทันทีครับ